Day 127 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 19, Day 2 reference

Watch: 86 - Applications of Autoencoders - Denoising using MNIST dataset (DigitalSreeni) | Lec18 Sparse and Denoising Autoencoders (Deep Learning For Visual Computing - IITKGP)

`day127_denoising_autoencoders.py`, segment by segment, in the order `main()` uses things. The damage functions, PSNR and the two baselines are new; the network and its training are Day 126's exercise parts, imported unchanged, with damage passed in through train_ae's noise hook. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: X = clean images, one row of 784 pixels each · D = damaged images · R = repaired images · M = where pixels were hidden · sigma, p = noise strength

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE); sys.path.insert(0, os.path.join(HERE, "..",
    "week-18-unsupervised-learning-i"))
with contextlib.redirect_stdout(io.StringIO()):
    import day126_exercise as d126
    import day126_autoencoders as d126m
    import day123_exercise as d123
    import day119_exercise as d119
ACTS = ["relu", "linear", "relu", "linear"]
SIZES = [784, 256, 32, 256, 784]
```

- `import day126_exercise as d126` — Day 126's network parts, exactly as the week's build will use them.
- `import day126_autoencoders as d126m` — only for its cached MNIST loader.
- `with contextlib.redirect_stdout(io.StringIO()):` — the exercise file prints its checks when imported; this keeps them out of today's output.

Segment 1 — Damage and scoring

The three kinds of damage

CODE

```
def gaussian(X, sigma, rng):
    return np.clip(X + rng.normal(0, sigma, X.shape).astype(np.float32), 0, 1)
def masking(X, p, rng):
    return X * (rng.uniform(size=X.shape) >= p).astype(np.float32)
def hide_square(X, size=10, rng=None):
    rng = rng or np.random.RandomState(0); Y = X.reshape(-1, 28, 28).copy(); M = np.zeros_like(Y,
        bool)
    for i, (r, c) in enumerate(rng.randint(4, 24 - size, size=(len(Y), 2))):
        Y[i, r:r + size, c:c + size] = 0; M[i, r:r + size, c:c + size] = True
    return Y.reshape(len(X), -1), M.reshape(len(X), -1)
```

- `np.clip(X + rng.normal(...), 0, 1)` — noise added, then pushed back into the valid range.

- `(rng.uniform(size=X.shape) >= p).astype(np.float32)` — a 0/1 mask: each pixel kept with probability 1 – p.
- `X.reshape(-1, 28, 28).copy()` — -1 lets NumPy work out the number of images; .copy() so the original is untouched.
- `Y[i, r:r + size, c:c + size] = 0` — two slices at once: a square block of rows and columns.

psnr, median3 and pca_denoise

MATH

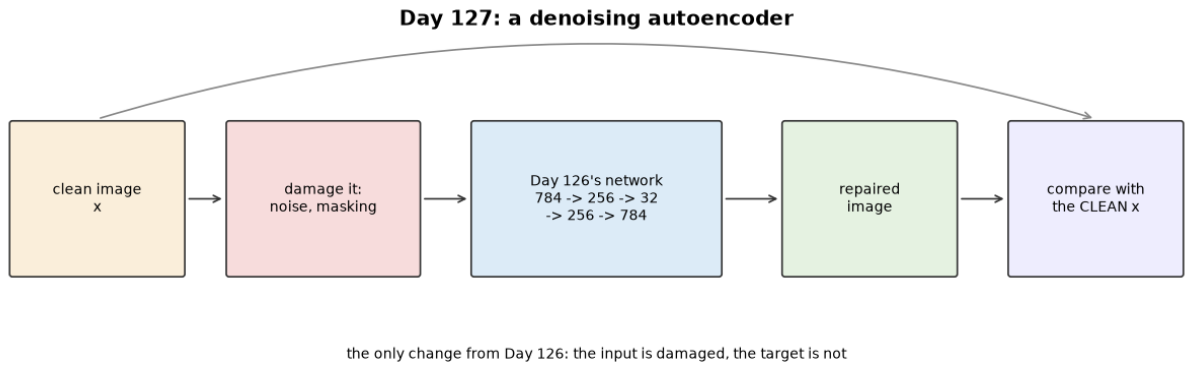
$$PSNR = 10 \log_{10}(1/MSE); \quad median3(x)_{ij} = \text{median of } x \text{ over rows } i-1..i+1 \text{ and columns } j-1..j+1$$

CODE

```
def psnr(R, X):
    return float(10 * np.log10(1.0 / np.mean((R - X) ** 2)))
def median3(X):
    P = np.pad(X.reshape(-1, 28, 28), ((0, 0), (1, 1), (1, 1)))
    stack = np.stack([P[:, i:i + 28, j:j + 28] for i in range(3) for j in range(3)])
    return np.median(stack, 0).reshape(len(X), -1)
def pca_denoise(Xn, pca, k=32):
    mu, V = pca
    return np.clip(d123.reconstruct(d123.transform(Xn.astype(np.float64), mu, V, k), mu, V, k), 0,
        1)
```

- `np.pad(..., ((0, 0), (1, 1), (1, 1)))` — a border of zeros, one pixel wide, around every image: no padding on the image axis.
- `[P[:, i:i + 28, j:j + 28] for i in range(3) for j in range(3)]` — nine shifted copies: each pixel's 3 x 3 neighbourhood, stacked.
- `np.median(stack, 0)` — the median across the nine copies, for every pixel of every image at once.

SYSTEM DIAGRAM



Damage in, clean target out.

Segment 2 — The experiments

demo_compare: three networks, one hook

CODE

```
plain, t1 = train(Xtr, Xte, None)
dae_g, t2 = train(Xtr, Xte, lambda X, rng: gaussian(X, 0.3, rng))
dae_m, t3 = train(Xtr, Xte, lambda X, rng: masking(X, 0.5, rng))
```

- `lambda X, rng: gaussian(X, 0.3, rng)` — a two-argument function, as train_ae expects: the noise level is fixed inside it.
- `train(Xtr, Xte, None)` — None: no damage, so this is Day 126's plain autoencoder.

demo_levels: freezing a loop variable

CODE

```
for s in (0.1, 0.5):
    nets[s], _ = train(Xtr, Xte, lambda X, rng, s=s: gaussian(X, s, rng))
```

- `lambda X, rng, s=s: ...` — s=s stores the current s as a default; without it every lambda would see the loop's last value.
- `nets[s], _ = train(...)` — unpacking: keep the network, discard the training time.

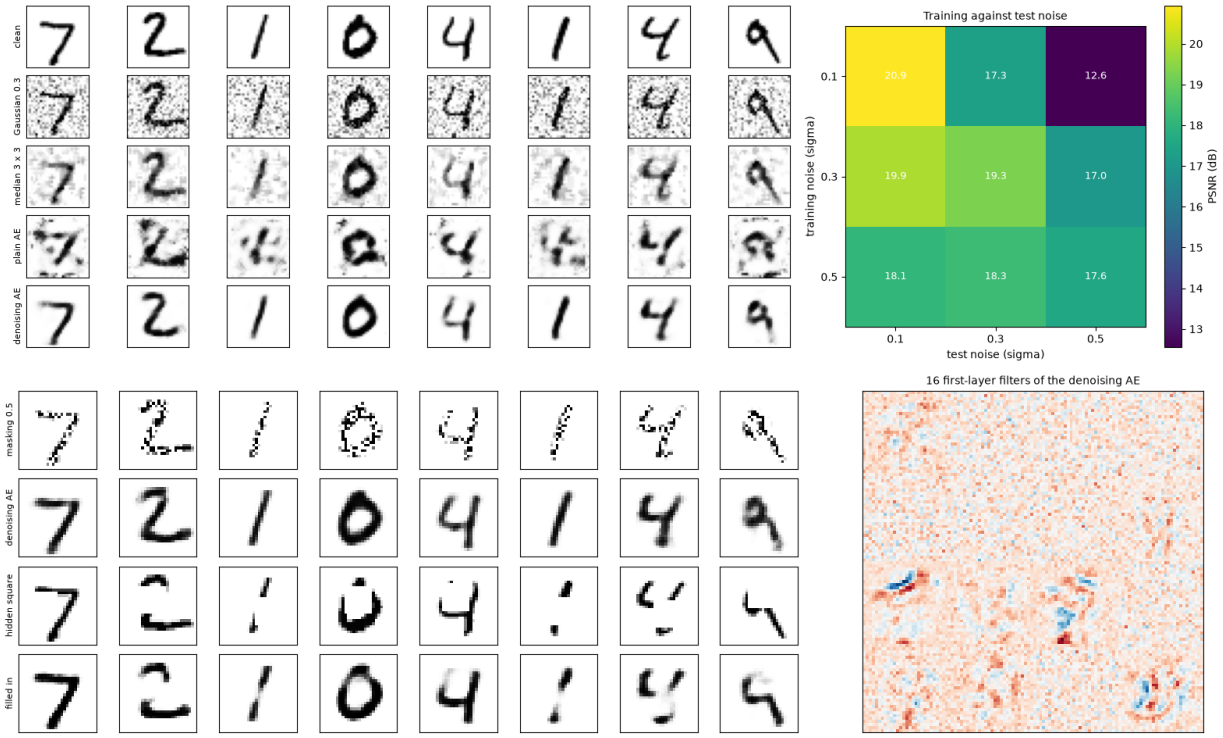
demo_square: scoring only inside the hidden square

CODE

```
D, M = hide_square(Xte); err = lambda R: float(np.sqrt(np.mean(((R - Xte)[M]) ** 2)))
```

- `(R - Xte)[M]` — boolean indexing with the mask: only the hidden pixels' errors are kept.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `np.clip(a, 0, 1)` — keep values inside a range.
- `np.pad(a, widths)` — add a border.
- `np.median(stack, 0)` — the median across the first axis.
- `lambda X, rng, s=s: ...` — freeze a loop variable in a function.
- `a[mask]` — the elements where the mask is True.
- `reshape(-1, 28, 28)` — let NumPy fill in one dimension.

End of Day 127 syntax reference. For the experiments and what they showed, see the Day 127 complete guide.